

Artificial Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

1.1 Fundamental Linear Algebra

- **Matrix**: A rectangular array of numbers, symbols, or expressions arranged in rows and columns.
- **Linear Transform**: A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c .
- **Matrix Multiplication**: An operation that combines two matrices to produce a third matrix, where the element at row i and column j of the resulting matrix is the dot product of the i -th row of the first matrix and the j -th column of the second matrix. The inner dimensions must match.
- **Matrix Inverse**: For a square matrix A , its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Transposition**: An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T .
- **Derivative**: A measure of how a function changes with respect to a change in its input.
- **Partial Derivative**: A derivative of a function of multiple variables with respect to one of those variables, holding the others constant.
- **Sigmoid Function**: A mathematical function having a characteristic "S"-shaped curve, often used as an activation function in neural networks. Its derivative is shown in slide 21.
- **Matrix Factorization**: A technique that decomposes a large matrix into the product of two smaller matrices, revealing latent factors that capture underlying relationships in the data.
- **Dimensionality Reduction**: The process of reducing the number of variables in a dataset while retaining as much important information as possible. This is often done to simplify data, improve visualization, or remove irrelevant features.
- **Low Rank Matrix Approximations**: Approximating a high-rank matrix with a lower-rank matrix, effectively reducing the dimensionality of the data while preserving essential information.

1.2 Machine Learning Foundations

- **Held Out Data**: A subset of the training data used to tune hyperparameters of a model, preventing overfitting to the training data.
- **Test Data**: A subset of the data used to evaluate the performance of a trained model on unseen data, providing an unbiased estimate of generalization ability.
- **Training Data**: The subset of data used to train a machine learning model, allowing the model to learn patterns and relationships from the data.

- **Hyperparameters:** Parameters that control the learning process of a model, such as learning rate, regularization strength, or number of layers in a neural network. They are not learned from the data but are set by the user.
- **Regularization:** A technique used to prevent overfitting by adding a penalty term to the loss function, discouraging overly complex models with large weights.
- **Learning Curves:** Graphs that plot the training and testing performance of a model as a function of the training set size or model complexity, providing insights into model bias and variance.
- **S-fold Cross Validation:** A model evaluation technique where the data is split into S folds, and each fold is used once as a validation set while the remaining folds are used for training. The average performance across all folds is used to estimate the model's generalization ability.
- **Data Analysis Pipeline:** A sequence of steps involved in analyzing data, including data collection, model design, parameter inference, and model application. Each step has potential sources of error that need to be considered.
- **Baseline Predictor:** A component of a recommender system that provides a basic prediction of a user's rating for an item, based on overall average ratings and user/item biases.
- **Bias-Variance Tradeoff:** The trade-off between model complexity and prediction accuracy. A model that is too complex may overfit the training data, leading to high variance and poor generalization to new data. A model that is too simple may underfit the data, leading to high bias and poor accuracy.

1.3 Supervised Learning Algorithms

- **LASSO:** A type of regularization that uses the L1 norm of the weights as the penalty term in the loss function, leading to sparsity in the model's weights.

1.4 Neural Networks

- **Perceptron:** A fundamental unit in neural networks that takes multiple inputs, applies weights to them, sums the weighted inputs, and produces an output based on a threshold.

1.5 Search Algorithms

- **Minimax Search:** A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally.
- **Alpha-Beta Pruning:** A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Game Tree:** A tree-like structure representing all possible sequences of moves in a game, with leaf nodes representing the outcomes.
- **Recursion:** A process where a function calls itself, eventually terminating in a base case.
- **Base Case:** The simplest form of a problem in a recursive approach, providing a direct solution without further recursion.
- **Recursive Case:** In a recursive approach, the same problem/method/data structure, but smaller, closer to the base case.
- **Breadth First:** A tree traversal method that visits nodes level by level.
- **Depth First:** A tree traversal method that traverses down a branch of a tree.
- **Binary Search Tree:** A tree data structure where each node has a value, its left child has a smaller value, and its right child has a larger value.

- **Search Tree:** A tree structure where each node represents a possible state in a search process.
- **Problem Solving as Search:** An approach to problem-solving that involves exploring possibilities to find a solution.
- **States:** Configurations of an environment in a search problem.
- **Actions:** Activities that move from one state to another in a search problem.
- **Search Algorithm:** Determines which actions should be tried in which order to find a solution in a search problem.
- **Successor Function:** A function that maps a state to its possible successor states.
- **World State:** Includes every detail of the environment in a search problem.
- **Search State:** Keeps only the details needed for planning (abstraction) in a search problem.
- **Rational Agent:** An agent whose actions maximize goal achievement, given available information.
- **Rational Behavior:** Doing the things which are expected to maximize goal achievement, given available information.

1.6 Artificial Intelligence (General)

- **Turing Test:** An operational test for intelligent behavior where a human interrogator tries to distinguish between a human and a machine based on their text-based responses.
- **Reinforcement Learning:** A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.

1.7 Logic and Reasoning

- **Inductive Reasoning:** The process of deriving general principles from specific observations.
- **Linear Regression:** A machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.
- **Model:** A mathematical representation of a system or process, used to make predictions or understand relationships between variables.
- **Overfitting:** A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to unseen data.
- **Loss Function:** A function that quantifies the difference between the predicted and actual values in a machine learning model.
- **Decision Boundary:** A line or hyperplane that separates different classes of data points in a classification problem.
- **Stochastic Gradient Descent:** An iterative optimization algorithm that updates model parameters using the gradient calculated from a randomly selected subset of the training data.
- **Logistic Regression:** A statistical model that uses a logistic function to model the probability of a binary outcome.
- **Perceptron Algorithm:** An iterative algorithm used for binary classification that updates the weights of a linear model based on misclassified data points.
- **Empirical Risk Minimizer:** A learning principle that aims to find the model that minimizes the average loss over the training data.

- **Binary Classification:** A type of classification problem where the goal is to assign data points to one of two classes.
- **Gradient Descent:** An iterative optimization algorithm that updates model parameters by moving in the direction of the negative gradient of the loss function.

1.8 Recommender Systems

- **Embeddings:** A low-dimensional representation of data points, often obtained through dimensionality reduction techniques like matrix factorization. They project high-dimensional data into a lower-dimensional space while preserving relationships between data points.
- **Collaborative Filtering:** A recommender system technique that predicts user preferences based on the preferences of similar users.
- **Recommender Systems:** Systems that predict user preferences for items, such as movies, products, or news articles, based on past user behavior and other relevant data.
- **Netflix Prize:** A competition launched by Netflix in 2006 to improve the accuracy of its movie recommendation system by offering a *1millionprizeforasignificantimprovementinpredictionaccuracy*.

1.9 Data Analysis and Bias

- **EigenSNPs:** Eigenvectors obtained from Principal Component Analysis (PCA) applied to Single Nucleotide Polymorphisms (SNPs) data. They represent directions of maximum variance in the genetic data, capturing patterns of genetic variation across populations.
- **Bias:** Systematic and repeatable errors in a data set or algorithm that can lead to unfair or discriminatory outcomes.
- **Fairness:** The property of a data set or algorithm that ensures equitable treatment of all individuals or groups, without discrimination.
- **Disparate Treatment:** A type of discrimination where decisions are (partly) based on a subject's protected attribute information.
- **Disparate Impact:** A type of discrimination where outcomes disproportionately hurt (or benefit) people with certain sensitive attribute values.
- **Proxies:** Seemingly neutral data points that can act as stand-ins for discriminatory factors, perpetuating bias in algorithms.
- **Anonymized Data:** Data that has had identifying information removed. However, this does not always guarantee privacy or prevent re-identification.
- **Interpretable Results:** Results from an AI algorithm that are easily understandable and explainable, increasing trust and transparency.
- **Group Fairness:** A fairness criterion that aims to equalize outcomes for different groups, ensuring that the probability of a particular outcome is similar across groups.
- **Individual Fairness:** A fairness criterion that aims to treat similar individuals similarly, regardless of group membership.
- **Automatic Prediction System:** A system that uses algorithms to make predictions automatically, often used in various applications like web advertising, lending, and ridesharing.

2 Problem Types

2.1 Foundational AI Concepts

- **Representing Recursion:** This problem focuses on understanding and applying the concept of recursion in AI, including its use in methods and data structures like linked lists and trees. Examples include the Fibonacci sequence and tree traversal.
- **Problem Solving as Search:** This problem explores the representation of problems as search spaces, defining states, actions, and search algorithms to find solutions. Examples include mazes, the 8-puzzle, and pathfinding on a map.
- **Optimizing Recursive Algorithms:** This problem addresses the efficiency of recursive algorithms by identifying and removing redundant calculations. The Fibonacci sequence is used as an example to illustrate the optimization process.
- **Representing Problems as Graphs:** This problem focuses on representing problems using graph structures, considering multiple paths to a solution and the implications for search algorithms. Examples include the 8-puzzle and pathfinding problems.
- **Defining State Spaces:** This problem involves defining appropriate state spaces for AI problems, distinguishing between world states (containing all details) and search states (containing only relevant details for planning). The Pac-Man game is used as an example.

2.2 AI Ethics and Safety

- **Identifying and Mitigating Bias in Data and Algorithms:** This problem focuses on identifying and addressing biases present in datasets and the resulting algorithms, which can lead to unfair or discriminatory outcomes. Examples include bias in hiring algorithms, loan applications, and criminal justice risk assessments.
- **Ensuring Data Privacy and Security:** This problem addresses the challenges of protecting sensitive personal data used in AI systems, while also ensuring the security of these systems from malicious attacks. Examples include anonymization techniques and the trade-offs between privacy and utility.
- **Balancing Accuracy and Interpretability in AI Models:** This problem explores the trade-off between the accuracy of AI models and their interpretability. Highly accurate models may be difficult to understand, making it hard to trust their outputs or identify potential biases. Examples include the choice between neural networks and logistic regression in medical diagnosis.
- **Defining and Achieving Fairness in AI:** This problem focuses on defining and operationalizing fairness in AI systems, considering different notions of fairness (e.g., group fairness, individual fairness) and their potential conflicts. Examples include the challenges of ensuring fairness in predictive policing and recidivism prediction.

2.3 Machine Learning Fundamentals

- **Understanding Perceptrons:** A perceptron, a fundamental building block of neural networks, is introduced, illustrating its structure and function.
- **Calculus in AI:** The slide highlights the importance of calculus, specifically derivatives, partial derivatives, and finding maxima/minima/gradients, in AI algorithms.
- **Calculating the Derivative of the Sigmoid Function:** The slide shows the step-by-step derivation of the derivative of the sigmoid function using calculus rules.
- **Overfitting:** A model that performs well on training data but poorly on unseen data, indicating it has learned the noise in the training data rather than the underlying patterns.

- **Underfitting:** A model that performs poorly on both training and test data, indicating it is too simple to capture the underlying patterns in the data.
- **Diagnosing Overfitting:** Comparing the performance of a model on training and test data to identify overfitting, where a large discrepancy indicates overfitting.
- **Hyperparameter Tuning:** The process of adjusting hyperparameters (parameters that control the learning process, not learned from data) to optimize model performance. This often involves using a held-out validation set.
- **Controlling Model Complexity:** Techniques to prevent overfitting by limiting the model's capacity to fit noise in the training data. This can involve limiting polynomial degree, using regularization, or other methods.
- **Training/Test Cycle:** The process of splitting data into training, validation (held-out), and test sets to train, tune, and evaluate a model, respectively. The test set is never used during training or tuning.
- **S-fold Cross Validation:** A technique to evaluate model performance and tune hyperparameters by repeatedly splitting the data into training and validation sets, using each part as a validation set once.
- **Data Analysis Pipeline:** The complete process of collecting, designing, inferring parameters for, and applying a model to data, highlighting potential issues at each stage.
- **Learning Curves:** Graphs showing the relationship between model complexity and accuracy on training and test sets, used to diagnose underfitting and overfitting.
- **Bias-Variance Tradeoff:** The balance between model simplicity (low variance, high bias) and model complexity (high variance, low bias), aiming for a model that generalizes well to unseen data.

2.4 Computer Vision

- **Image Classification Challenges:** This problem focuses on the difficulties in accurately classifying images due to variations in viewpoint, illumination, deformation, occlusion, and background clutter.
- **Representing Images Numerically:** This problem addresses the challenge of representing images as numerical data that can be processed by computers, highlighting the semantic gap between human perception and computer representation.
- **Finding Invariant Features:** This problem explores the need to identify features in images that remain consistent across different variations in viewpoint, lighting, and other factors, crucial for robust image classification.
- **Bag of Features Approach:** This problem focuses on using the Bag of Features method for image classification, which involves extracting local features, creating a visual vocabulary, and representing images as frequency distributions of visual words.
- **Noise Reduction in Images:** This problem addresses the challenge of removing noise from images, exploring techniques like mean filtering and moving averages to smooth out noisy pixel values.
- **Image Sharpening:** This problem focuses on enhancing image details by reversing the effects of blurring, using mathematical formulas and filters to increase the sharpness and clarity of images.
- **Document Representation for Retrieval:** This problem explores representing documents numerically using word frequencies for efficient information retrieval.

2.5 Linear Algebra for AI

- **Representing Linear Transformations with Matrices:** Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic:** The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for many AI computations.
- **Solving Systems of Linear Equations using Matrices:** Systems of linear equations can be represented and solved efficiently using matrix operations, particularly using matrix inverses.
- **Matrix Transposition:** Matrix transposition is a fundamental operation that involves swapping rows and columns of a matrix, useful for data manipulation in AI.
- **Parallelizing Matrix Multiplication:** Domain decomposition is presented as a method for parallelizing matrix multiplication, improving computational efficiency in AI.
- **Matrix Factorization:** Approximating a high-dimensional data matrix as a product of two lower-dimensional matrices to reveal latent factors.
- **Dimensionality Reduction:** Transforming data from a higher number of dimensions to a lower number of dimensions to discard irrelevant features, visualize high-dimensional data, or reflect the intrinsic dimensionality.
- **Low Rank Matrix Approximations:** Reducing the rank of a matrix to achieve dimensionality reduction, using equivalent definitions based on linearly independent subsets of columns/rows or matrix factorization.

2.6 Recommender Systems

- **Recommender System Algorithms:** Inferring missing ratings in a user-item matrix based on the ratings of similar users.
- **Collaborative Filtering:** Predicting a user's rating for an item based on the ratings of similar users.
- **Modeling Systematic Biases:** Accounting for biases in user ratings and movie ratings to improve the accuracy of recommender systems.
- **Dealing with Missing Data in Recommender Systems:** Addressing the challenge of missing entries in the user-item rating matrix by replacing missing values or optimizing reconstruction error over known entries.
- **Clustering Data:** Grouping a set of objects into classes of similar objects, where objects within a cluster are similar and objects from different clusters are dissimilar. Examples include clustering documents, images, genes, and movies.
- **Evaluating Clustering Results:** Assessing the "goodness" of resulting clusters, either subjectively (visual inspection of proximity matrices and PCA-reduced visualizations) or objectively (using known class labels and metrics like purity).
- **Understanding Distance Measures:** Using appropriate distance measures (Euclidean distance for continuous data, Hamming distance for discrete data) to quantify the similarity or dissimilarity between data points.

3 Algorithm Solutions

3.1 Linear Algebra

- **Matrix Addition:** $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication:** $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication:** $C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$
- **Matrix Inverse:** $AA^{-1} = A^{-1}A = I$
- **Solving Linear Equations using Matrix Inverses:** $Ax = bx = A^{-1}b$

3.2 Machine Learning Fundamentals

- **Multiple Linear Regression:** Find the optimal coefficients β that minimize the sum of squared errors between the observed and predicted values. This is achieved by solving the equation $\beta = (X^T X)^{-1} X^T y$, where y is the vector of observed values, X is the design matrix of independent variables, and β is the vector of coefficients.
- **Regularized Linear Regression:** $E(w) = RSS(w) + \lambda R(w)$, where $R(w)$ is either $\|w\|_2^2$ or $\|w\|_1$, and λ controls the trade-off between training error and complexity.
- **S-fold Cross Validation:** Split the training data into S equal parts; use each part as a validation dataset and the others as a training dataset; choose the hyperparameter leading to the best average performance.
- **Fairness through Blindness:** Ignore protected attributes during model training and prediction.
- **Group Fairness:** $P(outcome|S) = P(outcome|T)$, where S and T are two groups. This is insufficient as a fairness metric.
- **Individual Fairness:** Treat similar individuals similarly; similar individuals should have similar outcomes. How to achieve this is an open research question.
- **Detecting and Removing Bias from Models:** Keep bias features in the data during training, then remove what was learned from these features after training.

3.3 Classification Algorithms

- **Perceptron Algorithm:** Iteratively update the weight vector w by adding $y_n x_n$ if the data point x_n is misclassified (i.e., $\text{sign}(w^T x_n) \neq y_n$).
- **Bag of Features:** Represent images by frequencies of "visual words" (Bags of features)
- **Nearest Neighbor Classifier:** Assign label of nearest training data point to each test data point

3.4 Neural Networks

- **Derivative of Sigmoid Function:** $\frac{d}{dx} \sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2}$
- **ReLU Activation Function:** A piecewise linear activation function defined as $f(x) = \max(0, x)$, introducing non-linearity into the network.
- **Backpropagation Algorithm:** An algorithm used to train neural networks by calculating the gradient of the loss function with respect to the network's weights and updating the weights iteratively to minimize the loss.
- **Max Pooling:** A pooling operation that selects the maximum value within a defined region (e.g., 2x2) of a feature map, reducing the spatial dimensions while preserving the depth.

- **Average Pooling:** A pooling operation that calculates the average value within a defined region (e.g., 2x2) of a feature map, reducing the spatial dimensions while preserving the depth.
- **LeNet-5 Architecture:** A specific CNN architecture designed for the MNIST dataset, consisting of convolutional layers, pooling layers, and fully connected layers.

3.5 Image Processing

- **Moving Average Filter:** Replace each pixel with an average of all the values in its neighborhood
- **Weighted Moving Average Filter:** Add weights to the moving average to give different importance to neighboring pixels
- **Image Sharpening:** $f_{sharp} = f + \alpha(f - f_{blur})$ where f is the original image, f_{blur} is the blurred image, and α is a scaling factor
- **Finite Difference Method:** $\frac{df}{dx}[x, y] \approx F[x + 1, y] - F[x, y]$
- **Image Gradient Magnitude:** $\|\nabla f\| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$
- **Image Gradient Direction:** $\theta = \tan^{-1}(\frac{\partial f}{\partial x} / \frac{\partial f}{\partial y})$
- **Anisotropic Gaussian:** $G_{\sigma_x, \sigma_y}(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \cdot e^{-\frac{1}{2}((\frac{x^2}{\sigma_x^2}) + (\frac{y^2}{\sigma_y^2}))}$
- **Isotropic Gaussian:** $G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \cdot e^{-\frac{x^2 + y^2}{2\sigma^2}}$

3.6 Search Algorithms

- **Minimax Search:** A recursive algorithm for two-player zero-sum games that explores the game tree to find the best move for the current player, assuming the opponent also plays optimally. It alternates between maximizing the score for the current player and minimizing the score for the opponent.
- **Alpha-Beta Pruning:** An optimization technique for Minimax search that reduces the search space by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha and beta values to prune branches.
- **Recursive Binary Search Tree Search:** Recursively check the current node's value (V) against the target value (X). If $V == X$, return 'True'. If $X < V$, recursively search the left subtree; if $X > V$, recursively search the right subtree. If no children exist, return 'False'.
- **Breadth-First Search:** Explore nodes level by level, adding children to a queue.
- **Depth-First Search:** Explore a branch completely before moving to another branch.

3.7 Recommender Systems

- **Matrix Factorization for Recommender Systems:** Approximates a user-item rating matrix as a product of two lower-dimensional matrices representing user and item latent factors. $r_{i,j} \approx u_i \cdot v_j$, where $r_{i,j}$ is the rating of user i for item j , u_i is the user's latent factor vector, and v_j is the item's latent factor vector.
- **Modeling Systematic Biases in Recommender Systems:** $r_{ij} \approx \mu + b_i + b_j + u_i \cdot v_j$, where r_{ij} is the rating, μ is the overall mean rating, b_i is the user bias, b_j is the item bias, and $u_i \cdot v_j$ represents user-item interactions.

3.8 Clustering Algorithms

- **K-means Clustering:** The algorithm iteratively assigns data points to the nearest cluster centroid, then recomputes the centroids as the mean of the points in each cluster, repeating until convergence.
- **Euclidean Distance:** $d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$, where x and y are p -dimensional vectors.
- **Hamming Distance:** Counts the number of differing bits between two binary vectors.
- **Matching Coefficient:** Hamming distance normalized by the number of attributes/bits.
- **Purity:** $\text{purity}(C, G) = \frac{1}{N} \sum_{i=1}^K \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of points of class j in cluster i .

3.9 SAT Solving

- **DPLL Algorithm:** A backtracking-based algorithm for solving the Boolean satisfiability problem (SAT). It systematically assigns truth values to variables, checking for contradictions and backtracking when necessary.
- **Conversion to CNF:** A set of rules to transform any logical expression into an equivalent Conjunctive Normal Form (CNF) expression. This is crucial for using SAT solvers, as many solvers are designed to work specifically with CNF.
- **DPLL:** A recursive algorithm that improves upon the Naïve SAT algorithm by incorporating Boolean Constraint Propagation (BCP) to efficiently identify and resolve unit clauses, signifying a sub-algorithm within DPLL that repeatedly identifies and assigns values to literals in unit clauses (clauses with only one unassigned literal).
- **Resolution Rule:** An inference rule that derives a new clause from two existing clauses containing a literal and its negation. The new clause combines the remaining literals from the original clauses using logical OR.
- **Clause Learning:** A technique used to learn from conflicts encountered during the DPLL search. When a conflict occurs, a concise reason (a clause) is learned to prevent similar conflicts in the future, improving efficiency.
- **Hamiltonian Path Encoding:** A method to encode the Hamiltonian Path Problem into a SAT problem. It uses Boolean variables h_{ij} representing whether the i -th vertex visited is vertex j . Constraints are added to ensure the path uses only existing edges and visits each vertex exactly once.
- **Graph Coloring Encoding:** A method to encode the Graph Coloring Problem into a SAT problem. It uses Boolean variables to represent the assignment of colors to vertices, with constraints ensuring that adjacent vertices have different colors.

3.10 Dynamic Programming

- **Recursive Fibonacci:** $\text{fib}(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & \text{otherwise} \end{cases}$